

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. Introdução

2. Instalação

A primeira etapa deste projeto consistiu na instalação e configuração inicial do MINIX na máquina virtual. Para isso, foi necessário seguir o manual de instruções apresentado no enunciado. Imagens da instalação podem ser encontradas nas Figuras 1 e 2.

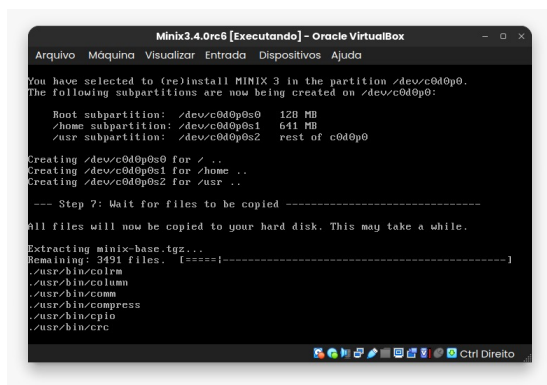


Figura 1. Etapa 7 de instalação do MINIX.

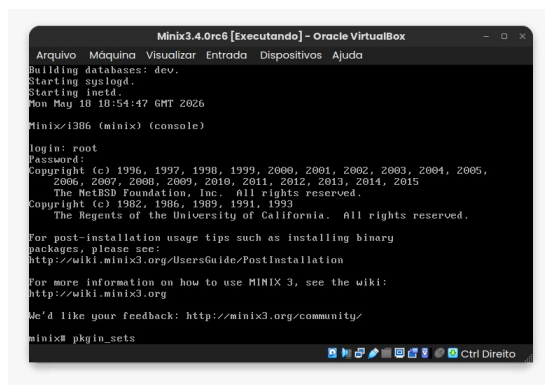


Figura 2. Atualização dos pacotes do MINIX.

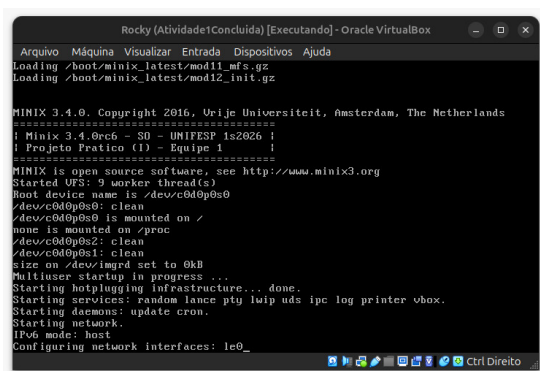
Além da instalação do MINIX, também foi necessário fazer a configuração de rede no *Virtual Box*, ajustando a política de endereçamento MAC para incluir apenas os endereços MAC de placas em rede NAT.

Além disso, foi necessário fazer o *clone* do repositório no ambiente virtual, para que possamos fazer as modificações que serão detalhadas em seções futuras deste relatório. Para isso, foi necessário configurar um Token de Acesso Pessoal do Github, o que nos permitiu executar os comandos do *git* no ambiente virtual.

Por fim, faz-se necessário ressaltar as configurações de hardware da máquina virtual. Para a execução deste projeto utilizamos uma máquina virtual com 1 *core* de processador, 1GB de memória RAM e 4GB de HD dinamicamente alocado.

3. Mudança nos Banners

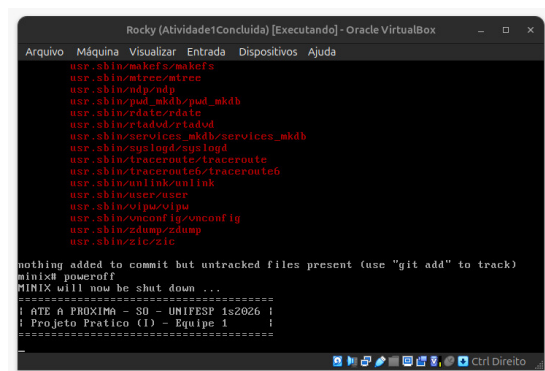
Para a mudança no banner durante *boot* e *poweroff*, foi necessário navegar no diretório `minix/minix/kernel/main.c`. Na função `announce()`, foram acrescentados *prints* para reproduzir o banner indicado no enunciado. Além disso, na função `prepare_shutdown()`, também inserimos *prints* para a impressão do banner. Os resultados podem ser encontrados nas Figuras 3 e 4.



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo Máquina Visualizar Entrada Dispositivos Ajuda
Loading /boot/minix_latest/mod11_mfs.gz
Loading /boot/minix_latest/mod12_init.gz

MINIX 3.4.0. Copyright 2016,rije Universiteit, Amsterdam, The Netherlands
=====
! Minix 3.4.0rc6 - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
MINIX is open source software, see http://www.minix3.org
Started VFS: 9 worker thread(s)
Root device name is /dev/c0d0p0s0
/dev/c0d0p0s0: clean
/dev/c0d0p0s0 is mounted on /
none is mounted on /proc
/dev/c0d0p0s2: clean
/dev/c0d0p0s1: clean
size on /dev/ingrd set to 0kB
Multiuser startup in progress ...
Starting hotplugging infrastructure... done.
Starting services: random lance pty lwp uds ipc log printer vbox.
Starting daemons: update cron.
Starting network.
IPv6 mode: host
Configuring network interfaces: le0_
```

Figura 3. Resultado do banner de *boot*.



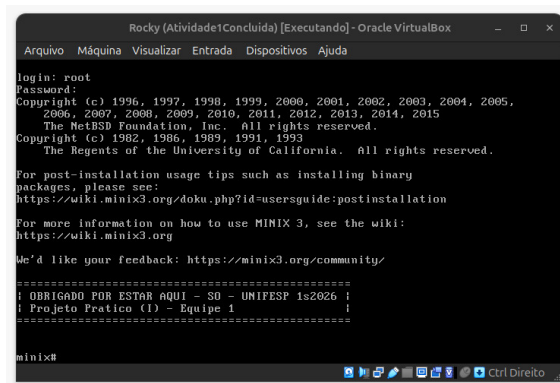
```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo Máquina Visualizar Entrada Dispositivos Ajuda

usr/sbin/makefs/makefs
usr/sbin/etree/etree
usr/sbin/ndp/ndp
usr/sbin/pad_mkdb/pad_mkdb
usr/sbin/rdate/rdate
usr/sbin/etad/rdate
usr/sbin/services_mkdb/services_mkdb
usr/sbin/syslogd/syslogd
usr/sbin/traceroute/traceroute
usr/sbin/traceroute6/traceroute6
usr/sbin/unlink/unlink
usr/sbin/user/user
usr/sbin/vmconfig/vmconfig
usr/sbin/zdump/zdump
usr/sbin/zic/zic

nothing added to commit but untracked files present (use "git add" to track)
minix# poweroff
MINIX will now be shut down ...
=====
! AT E PROXIMA - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
```

Figura 4. Resultado do banner de *poweroff*.

Por fim, para a modificação do banner de *login*, foi necessário navegar pelos diretórios `minix/etc/motd`, onde inserimos o banner indicado pelo enunciado. O resultado da modificação pode ser encontrado na Figura 5



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo Máquina Visualizar Entrada Dispositivos Ajuda

login: root
Password:
Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015
The NetBSD Foundation, Inc. All rights reserved.
Copyright (c) 1982, 1986, 1989, 1991, 1993
The Regents of the University of California. All rights reserved.

For post-installation usage tips such as installing binary
packages, please see:
https://wiki.minix3.org/doku.php?id=usersguide:postinstallation

For more information on how to use MINIX 3, see the wiki:
https://wiki.minix3.org

We'd like your feedback: https://minix3.org/community/

=====
! OBRIGADO POR ESTAR AQUI - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====

minix#
```

Figura 5. Resultado do banner de *login*.

4. Mudança `exec.c`

Para registrar a execução de processos exibindo o comando e seu caminho absoluto, a implementação foi dividida entre o *Virtual File System* (VFS) e o *Process Manager* (PM), respeitando a modularidade da arquitetura do MINIX [MINIX 3 Project].

Inicialmente, adicionou-se o campo `execpath` à estrutura compartilhada `exec_info` (no arquivo `libexec.h`) como um campo de comunicação entre o VFS e o PM. No VFS (`vfs/exec.c`), logo após o carregamento do executável, o caminho absoluto armazenado na variável `finalexec` é copiado para `execi.args.execpath` utilizando a função `strncpy`. Em seguida, o VFS aciona o PM por meio de `libexec_pm_newexec`.

A exibição da mensagem foi centralizada no PM (`pm/exec.c`). Na rotina `do_newexec`, os dados preenchidos pelo VFS são recebidos via `sys_datacopy`. Após a cópia dos argumentos para a memória do PM, inseriu-se um `printf` para exibir a mensagem Executando: `<caminho/comando>`. O resultado dessa modificação pode ser encontrado na Figura 6.

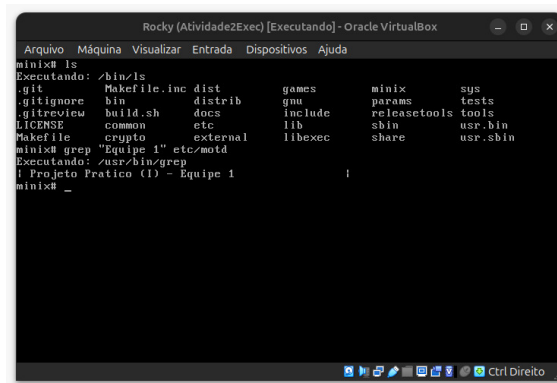


Figura 6. Resultado da modificação do `exec.c`

5. Descrição do Exercício Proposto

Este projeto tem como objetivo avaliar o desempenho de diferentes algoritmos de escalonamento no sistema operacional MINIX 3.4.0rc6. A metodologia consiste na análise teórica e na modificação do código-fonte do sistema para implementar três políticas de escalonamento distintas, além da abordagem padrão do MINIX. Para isso, foram implementados os algoritmos: Round-Robin, Múltiplas Filas e *First-Come, First-Served* (FCFS).

Para analisar o comportamento e a eficiência de cada algoritmo sob diferentes perfis de carga, serão conduzidos testes comparativos simulando cenários com processos intensivos em computação (*CPU-bound*) e intensivos em entrada/saída (*I/O-bound*). A avaliação quantitativa será realizada por meio do programa de testes `teste_processos.c`, variando-se a carga de trabalho em escalas de 10, 50, 100 e 200 processos simultâneos. Adicionalmente, o sistema será avaliado em diferentes cenários, submetendo os escalonadores à execução de 10 mil processos *I/O-bound* e 1 milhão de processos *CPU-bound*.

6. Descrição dos Algoritmos de Escalonamento

6.1. Algoritmo Padrão do MINIX

6.2. Round-Robin

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*), é um dos algoritmos mais simples e antigos devido à sua fácil implementação. Seu funcionamento gira em torno da definição de uma fatia de tempo de CPU (*quantum*), em que o processo que está na CPU terá um tempo limitado de uso e, ao final deste tempo, haverá preempção e um novo processo será escolhido para ser executado [Tanenbaum 2016].

A implementação concentrou-se em `minix/servers/sched/schedule.c`, sem alterar o *kernel*. O quantum é $\frac{1000}{n} ms$, com n dado por `count_ready_user_queue()` (processos de usuário prontos em `USER_Q`, via `sys_getinfo(GET_PROC)` e `proc_is_runnable()`). Foram incluídos `<minix/timers.h>` e `"kernel/proc.h"` e criadas `apply_variable_quantum()` e `reschedule_all_user_quantum()`; em `do_start_scheduling()`, `do_stop_scheduling()` e `do_noquantum()` o tempo de CPU é recalculado e aplicado com `sys_schedule()`. Processos de sistema (`is_system_proc`) mantêm prioridade própria; os restantes ficam fixos em `USER_Q`.

Para obter RR sem mudar `enqueue()`, `dequeue()` e `pick_proc()` (`minix/kernel/proc.c`), todos os usuários são forçados a `USER_Q` (7) em `do_start_scheduling()` e `do_nice()`; a rotação na cauda ao expirar o quantum ou ao desbloquear por E/S equivale ao *Round Robin* entre processos nessa fila. Em `minix/servers/pm/utility.c`, `nice_to_priority()` passa sempre `*new_q = USER_Q`, pelo que `nice()` deixa de distribuir processos pelas 16 filas.

Removeu-se a política MLFQ no SCHED em `do_noquantum()` eliminou-se `rmp->priority += 1`. `balance_queues()` e `init_scheduling()` ficaram vazias e em `main.c` deixou de se chamar `balance_queues()` nas notificações do CLOCK. Ao expirar o quantum, o processo permanece em `USER_Q`, recalcula a fatia com `apply_variable_quantum()` e reagenda-se com `schedule_process_local()` (reinserção na cauda no kernel via `sched_proc`).

6.3. Algoritmo a ser escolhido

7. Resultados Obtidos e Discussão

Para garantir a robustez estatística de nossos resultados, realizamos 5 execuções para cada algoritmo, e calculamos a média e o desvio-padrão de cada um deles. Ao final das execuções, foi confeccionado um gráfico de barras para cada classe de processos, no qual o eixo x representa o número de processos e o eixo y representa o tempo de execução médio.

Também é válido ressaltar que todos os testes de desempenho foram executados no mesmo computador, o que garante que os resultados são consistentes e não foram afetados por fatores externos. A configuração do computador utilizado foi um processador 12th Gen Intel(R) Core(TM) i5-12500H com 16GB de RAM, que estava com o Windows 11 instalado. Por fim, a versão da Oracle VirtualBox é a 7.2.6.

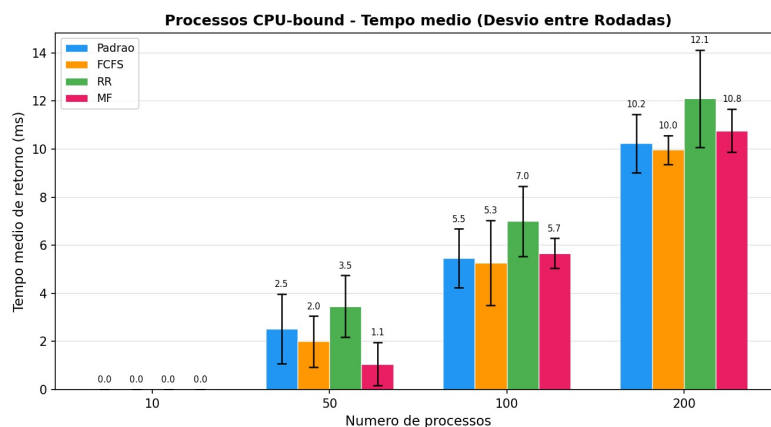


Figura 7. Resultados obtidos para processos *CPU-bound*

É possível verificar na Figura 7 que, com uma carga base de 10 processos, o tempo de retorno é aproximadamente 0.0 ms para todos os algoritmos. O comportamento dos algoritmos começa a divergir em 50 processos, onde o algoritmo MF se destaca com o menor tempo médio de resposta (1.1 ms) **pocoto escreve o motivo**. À medida que o sistema é levado a cenários de alta concorrência (100 e 200 processos), os algoritmos FCFS e Padrão demonstram melhor escalabilidade e estabilidade **justificar porque breno**. No teste mais extremo de 200 processos, o FCFS obtém a melhor marca com 10.0 ms, seguido rigorosamente pelo escalonador Padrão com 10.2 ms. **O desempenho superior do FCFS nestas condições se justifica teoricamente pela sua natureza não preemptiva: processos limitados por CPU executam até o fim sem interrupções forçadas, eliminando o overhead gerado por constantes trocas de contexto. breno verificar informacao** Em contrapartida, o algoritmo Round Robin (RR) sofre o maior impacto sob carga pesada, atingindo 12.1 ms e exibindo a maior variância (desvio-padrão) entre as rodadas. Isso ocorre devido ao número de processos na fila, pois o quantum com fração justa será muito menor ($\frac{1000}{200} = 5\text{ ms}$), dessa forma, um processo que em execução ficará pouco tempo na CPU, sendo necessário muitas trocas de contexto.

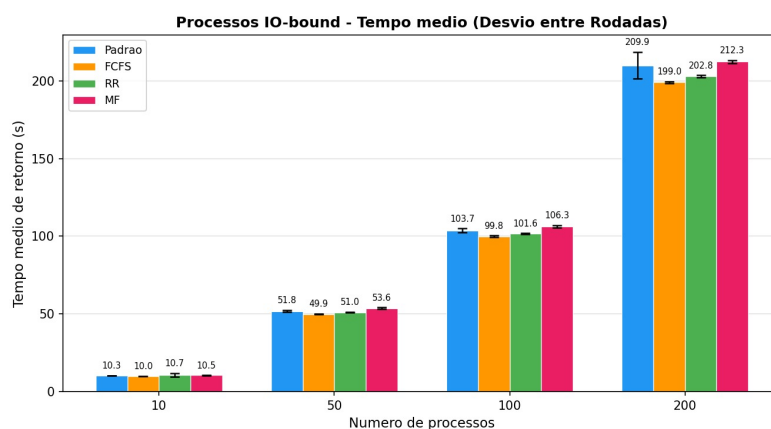


Figura 8. Resultados obtidos para processos *IO-bound*

No gráfico de processos IO-bound, os tempos de retorno operam na casa dos se-

gundos, refletindo o alto custo das operações de espera por dispositivos. Neste ambiente, o algoritmo FCFS apresenta sistematicamente a melhor performance em todas as faixas de teste, variando de 10.0 s com 10 processos até 199.0 s com 200 processos. O Round Robin (RR) acompanha o FCFS de perto como a segunda melhor alternativa para I/O intenso, fechando o pior caso com 202.8 s. Por outro lado, algoritmos baseados em prioridades dinâmicas mostraram-se menos favoráveis para este tipo exclusivo de carga. O algoritmo MF obteve o tempo de retorno mais lento no pico de estresse, marcando 212.3 s com 200 processos, e o Padrão apresentou a segunda pior marca neste mesmo nível (209.9 s). Essa degradação em algoritmos complexos (Padrão e MF) ocorre porque processos de I/O frequentemente bloqueiam e retornam à fila de prontos; a manipulação das múltiplas filas pode introduzir pequenos atrasos em relação a um esquema FIFO simples, onde o processo recém-acordado do I/O entra numa fila plana e, não havendo preempção de tempo no FCFS, rapidamente retoma o processador para emitir seu próximo comando de I/O, otimizando o paralelismo entre dispositivos e CPU.

Referências

- MINIX 3 Project. Minix 3 wiki documentation. <https://wiki.minix3.org/>. Accessed: 2026-06-04.
- Tanembaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.